

Solución — Examen Parcial 2

Ejercicio 1 — btree_son_iguales

Dos árboles son iguales si: (a) ambos son vacíos, o (b) las raíces son iguales según `cmp` y los subárboles izquierdo y derecho son respectivamente iguales.

```
int btree_son_iguales(BTree a1, BTree a2, FuncionComparadora cmp) {
    if (btree_empty(a1) && btree_empty(a2)) return 1;
    if (btree_empty(a1) || btree_empty(a2)) return 0;
    return cmp(a1->dato, a2->dato) == 0
        && btree_son_iguales(a1->left, a2->left, cmp)
        && btree_son_iguales(a1->right, a2->right, cmp);
}
```

Ejercicio 2 — ABB generales

a) bstree_cota_inferior

Versión iterativa. Se baja por el árbol manteniendo un candidato. Cuando el nodo actual es mayor que `x`, es un candidato y se baja a la izquierda buscando uno menor. Cuando es menor o igual, no sirve y se baja a la derecha.

```
void *bstree_cota_inferior(BSTree arbol, void *x,
    FuncionComparadora cmp) {
    void *candidato = NULL;
    while (arbol != NULL) {
        if (cmp(arbol->dato, x) > 0) {
            candidato = arbol->dato;
            arbol = arbol->left;
        } else {
            arbol = arbol->right;
        }
    }
    return candidato;
}
```

Versión recursiva. Si el dato actual es mayor que `x`, se busca uno mejor en el subárbol izquierdo; si no hay, el actual es la respuesta. Si el dato actual es menor o igual, se busca en el subárbol derecho.

```
void *bstree_cota_inferior(BSTree arbol, void *x,
    FuncionComparadora cmp) {
    if (arbol == NULL) return NULL;
    if (cmp(arbol->dato, x) > 0) {
        void *en_izq = bstree_cota_inferior(arbol->left, x, cmp);
        return en_izq != NULL ? en_izq : arbol->dato;
    }
    return bstree_cota_inferior(arbol->right, x, cmp);
}
```

Otra versión recursiva, arrastrando el candidato como parámetro. La función que se invoca:

```
void *bstree_cota_inferior(BSTree arbol, void *x,
    FuncionComparadora cmp) {
    return bstree_cota_inferior_aux(arbol, x, NULL, cmp);
}
```

La función que contiene la lógica, que arrastra el candidato como parámetro:

```
void *bstree_cota_inferior_aux(BSTree arbol, void *x, void *
    candidato,
                                FuncionComparadora cmp) {
    if (arbol == NULL) return candidato;
    if (cmp(arbol->dato, x) <= 0)
        return bstree_cota_inferior_aux(arbol->right, x,
            candidato, cmp);
    return bstree_cota_inferior_aux(arbol->left, x, arbol->dato,
        cmp);
}
```

b) Implementación de `persona_comparar_edad`:

```
int persona_comparar_edad(void *a, void *b) {
    if (!a && !b) return 0;
    if (!a) return -1;
    if (!b) return 1;
    return ((Persona *) a)->edad - ((Persona *) b)->edad;
}
```

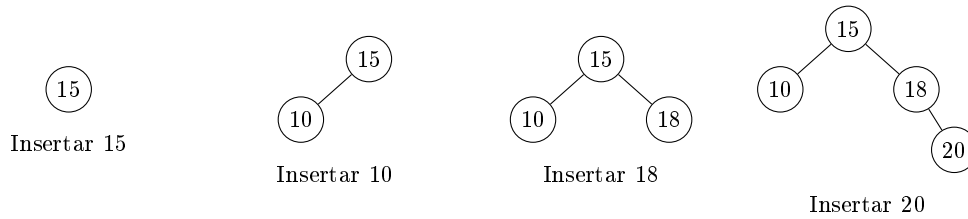
Fragmento que encuentra la persona de menor edad mayor a 18 años:

```
Persona referencia = {NULL, 18};
Persona *resultado = bstree_cota_inferior(arbol, &referencia,
    persona_comparar_edad);
```

Ejercicio 3 — Construcción del AVL

Secuencia de inserción: **15, 10, 18, 20, 25, 19, 17**.

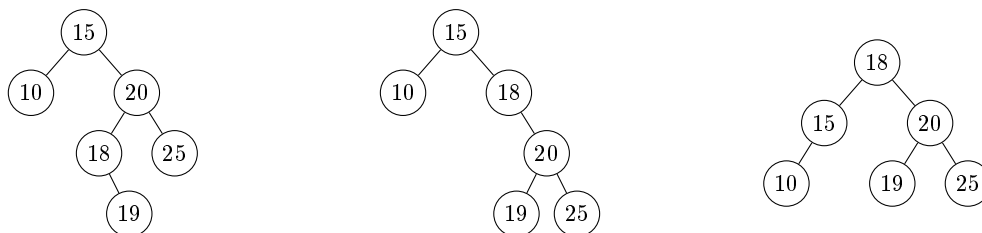
Pasos 1 a 4 (sin rebalanceo):



Paso 5. Insertar 25. El nodo 18 queda con factor de balance -2 (desbalance derecha-derecha): **rotación simple a izquierda en 18**.



Paso 6. Insertar 19. El nodo 15 queda con factor -2 y su hijo derecho (20) con factor $+1$ (caso **derecha-izquierda**): **rotación doble DI en 15**, que consiste en una rotación a derecha en 20 seguida de una rotación a izquierda en 15.



Paso 7. Insertar 17. Balanceado, no requiere rotación:

